# Experiments with Kafka

> [!NOTE] The full runnable setups live in the subdirectories referenced below. This file consolidates the insights that are documented there so the hand-in captures every experiment outcome in one place.

Project Link: https://github.com/cyrilgabriele/EDPO-Project-FS26/tree/main/assignments/ex-1

## Producer Experiments (`./producer-experiments`)

### Infrastructure and Code

- Based on the lab02Part3-ManyBrokers stack, trimmed to `controller`, `kafka1`, and `kafka2` as defined in `producer-experiments/docker-compose.yml`.
- `ClickStream-Producer` emits one event roughly every 150 ms with `acks=0`, `retries=0`, and no idempotence. `ClickStream-Consumer` continuously logs the payloads from `click-events`.

### Experiment — Message Loss with `acks=0` and `retries=0`

**Goal:** Show that disabling acknowledgments and retries results in permanent data loss when the leader crashes.

**Procedure:**

1. `cd assignments/ex-1/producer-experiments && docker compose up -d` to start controller + two brokers.
2. Launch the producer and consumer run configurations from IntelliJ (same binaries as in the directory README).
3. Determine the active leader via `kafka-topics --describe` and stop that broker (e.g., `docker stop producer-experiments-kafka1-1`).

**Observed behavior (see `./producer-experiments/producer-experiments.md`):**

- Producer log excerpt:

```
clickEvent sent: eventID: 75 ...
clickEvent sent: eventID: 76 ...
Current Leader: 2 host: localhost port: 9092
In Sync Replicates: [2]
clickEvent sent: eventID: 77 ...
```

- Consumer log excerpt skips the `eventID=76` record entirely even though the producer printed "sent".

**Insights:**

- With `acks=0`, the client "fire-and-forgets"; Kafka may never receive the record, yet the application cannot tell.

- `retries=0` eliminates the chance to resend while the leader is down, so the unlucky record disappears forever.
- Remediation requires at least `acks=all`, `retries > 0`, and ideally `enable.idempotence=true` (all described in the producer README) to regain durability without introducing duplicates.

## Consumer Experiments (`./consumer-experiments`)

### Shared Preparation

- Topic `click-events` with a single partition is created by the provided producer (`ClickStream-Producer/src/main/java/...`).
- Producer emits ordered `eventID` values `1..20` (and beyond) before each test.
- Consumers log every `eventID`. When expecting `auto.offset.reset` to apply, a fresh `group.id` is configured (`consumer.properties`).

### Experiment A — Safe Replay with `auto.offset.reset=earliest`

**Goal:** Confirm that when no committed offsets exist, `earliest` forces a full rewind.

**Procedure:**

1. Produce the first batch of events before starting the consumer.
2. Start the consumer with the defaults (`group.id=grp1`, `auto.offset.reset=earliest`).
3. Kill the consumer before it auto-commits (Ctrl+C within five seconds), then restart it unchanged.

**Observed behavior:**

```
Received click-events - value: {eventID=45, timestamp=175674327349750, xPosition=926,
Position=521, clickedElement=EL16}- partition: 0
Received click-events - value: {eventID=46, timestamp=175674494709208, xPosition=179,
Position=686, clickedElement=EL9}- partition: 0
Received click-events - value: {eventID=47, timestamp=175674656272000, xPosition=793,
Position=777, clickedElement=EL5}- partition: 0
Received click-events - value: {eventID=48, timestamp=175674814550250, xPosition=1076,
yPosition=1042, clickedElement=EL5}- partition: 0
^C%
```

- Shutdown while printing events `45–48` proves offsets were never committed.

```
events-0
[main] INFO org.apache.kafka.clients.consumer.internals.SubscriptionState - [Consumer
lientId=consumer-grp1-1, groupId=grp1] Resetting offset for partition click-events-0
 position FetchPosition{offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAnd
och{leader=Optional[localhost:9092 (id: 2 rack: null)], epoch=0}}.
Received click-events - value: {eventID=0, timestamp=175666893516333, xPosition=581,
osition=949, clickedElement=EL2}- partition: 0
Received click-events - value: {eventID=1, timestamp=175667097478041, xPosition=1835,
Position=246, clickedElement=EL3}- partition: 0
Received click-events - value: {eventID=2, timestamp=175667259613791, xPosition=182,
osition=826, clickedElement=EL18}- partition: 0
Received click-events - value: {eventID=3, timestamp=175667431334583, xPosition=1488,
```

- Restart log shows “Resetting offset … offset=0” and the consumer replays from `eventID=0` in order.

**Insights:**

- Missing commits + `earliest` guarantee deterministic replay, which is ideal for at-least-once processing and disaster recovery.
- Operationally, you must budget for the fact that a restart can take as long as processing the full backlog again.

### Experiment B — History Skipped with `auto.offset.reset=latest`

**Goal:** Show that switching to `latest` causes brand-new consumer groups to miss retained data if they have no offsets.

**Procedure:**

1. Keep producing to `click-events` and edit `consumer.properties` to use `group.id=grp2` and `auto.offset.reset=latest`.
2. Start the consumer, observe no records being printed initially, then produce more events.

**Observed behavior:**

```
[main] INFO org.apache.kafka.clients.consumer.internals.ConsumerCoordinator - [Consume
 clientId=consumer-grp2-1, groupId=grp2] Found no committed offset for partition click
events-0
[main] INFO org.apache.kafka.clients.consumer.internals.SubscriptionState - [Consumer
lientId=consumer-grp2-1, groupId=grp2] Resetting offset for partition click-events-0 t
 position FetchPosition{offset=90, offsetEpoch=Optional.empty, currentLeader=LeaderAnd
poch{leader=Optional[localhost:9094 (id: 3 rack: null)], epoch=0}}.
Received click-events - value: {eventID=90, timestamp=177609351241541, xPosition=987,
Position=155, clickedElement=EL13}- partition: 0
Received click-events - value: {eventID=91, timestamp=177609512017208, xPosition=375,
Position=530, clickedElement=EL12}- partition: 0
Received click-events - value: {eventID=92, timestamp=177609673399541, xPosition=1301,
```

- Client joins `grp2` with no offsets and immediately resets to offset 90.

```
Current Leader: 3 host: localhost port: 9094
In Sync Replicates: [3, 2]
clickEvent sent: eventID: 0, timestamp: 177594621786083, xPosition: 325, yPosition: 969,
ckedElement: EL14,
clickEvent sent: eventID: 1, timestamp: 177594814080041, xPosition: 1493, yPosition: 48
ickedElement: EL19,
clickEvent sent: eventID: 2, timestamp: 177594974046208, xPosition: 808, yPosition: 753,
ckedElement: EL11,
clickEvent sent: eventID: 3, timestamp: 177595135443041, xPosition: 1181, yPosition: 10
lickedElement: EL3,
clickEvent sent: eventID: 4, timestamp: 177595306025666, xPosition: 547, yPosition: 18,
kedElement: EL15,
clickEvent sent: eventID: 5, timestamp: 177595471604958, xPosition: 572, yPosition: 682,
ckedElement: EL4,
clickEvent sent: eventID: 6, timestamp: 177595638387458, xPosition: 69, yPosition: 526,
```

- Producer emits IDs `0..6` before the consumer prints anything; those remain unread by `grp2`.

**Insights:**

- `latest` is unsafe for analytics-style workloads that expect replay of retained data; you must explicitly use `earliest`, manual `seek`, or pre-seed offsets to prevent silent data gaps.
- Even though Kafka retained the skipped records, from the application's perspective the data is "lost," illustrating the misconfiguration risk discussed in the README.

## Fault Tolerance & Reliability (`./fault-tolerance-and-reliability`)

### Infrastructure and Components

- `docker-compose.yml` provisions one controller plus `kafka1`, `kafka2`, `kafka3` brokers with replication settings matching a three-node ISR, including `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=3`.
- `ClickStream-Producer` logs leader/ISR changes and records ACK/failure events while producing monotonic `eventID`s.
- `ClickStream-Consumer` has `enable.auto.commit=false`, explicitly commits checkpoints, and reports gaps via `GAP DETECTED` log lines.

### Tutorial 1 — Measuring Leader Failover Time

**Goal:** Quantify the pause between the last ACK before a broker crash and the first ACK after a new leader is elected.

**Procedure:**

1. Start the stack (`cd assignments/ex-1/fault-tolerance-and-reliability && docker compose up -d`).
2. Run producer and consumer.
3. Identify the leader via `kafka-topics --describe` and kill the corresponding container (`docker compose kill -s KILL kafka3`, etc.).

**Observed behavior:**

- Last ACK before failure: `eventID=212` at `18:12:51.570`.
- Leader switched (from broker `2` to `3`) logged at `18:13:00.375`.
- Producer metrics from the README: `lastAckToLeaderElectedMs=8805`, `lastAckToFirstRecoveredAckMs=8890`, `firstAckAfterRecoveryMs=85`.
- Consumer kept processing `211..271` without gaps once service resumed.

**Insights:**

- The ~8.9 s outage was purely availability-related; no ACKed data was lost.
- Downtime must be measured from the last successful ACK rather than from the first log line about leader election, as most of the pause happens before the detector fires.

### Tutorial 2 — True Loss with `acks=1`

**Goal:** Demonstrate that leader-only acknowledgments can still lose records if the leader crashes before replication.

**Procedure:**

1. Configure producer with `acks=1`, `retries=0` and keep the consumer running.
2. Kill the leader broker while load is active.

**Expected/Observed behavior (per README):**

- Some events around the failure window are logged as `ACKED` by the producer.
- The consumer later reports gaps for those IDs because the leader died before followers replicated them.

**Insights:**

- `acks=1` protects only against client-side buffering losses; it does not guarantee durability beyond the leader.
- Switching to `acks=all` plus retries/idempotence is mandatory when data loss is unacceptable.

### Tutorial 3 — Anti-pattern: `acks=all` with `min.insync.replicas=1`

**Goal:** Show that shrinking the ISR to one replica nullifies the benefits of `acks=all`.

**Procedure:**

1. Set `acks=all`, `retries=5` in the producer.
2. Reduce topic durability with `kafka-configs --alter --add-config min.insync.replicas=1`.
3. Stop two brokers, keep producing (ACKs still appear because the single ISR satisfies `acks=all`).
4. Kill the last broker and then restart the cluster.

**Expected/Observed behavior:**

- Messages acknowledged while only one ISR remained disappear after the final crash/restart, even though the producer logged success.

**Insights:**

- `acks=all` only waits for the current ISR; enforcing `min.insync.replicas>=2` is the real guardrail.
- Keeping `unclean.leader.election.enable=false` avoids further loss, but not when ISR has already collapsed.

## Evidence and Reproducibility

- Detailed command transcripts, log snippets, screenshots, and configuration files remain in each experiment directory referenced above.
- To reproduce, follow the step-by-step sections in the respective README files; they include docker compose commands, IntelliJ run configs, and cleanup commands (`docker compose down -v`).